Survey in Machine Translation Evolution

Barbalata Radu Ionut
Sapienza Universita' di Roma
barbalata.2002688@studenti.uniroma1.it

Abstract

This survey traces the evolution of Machine Translation (MT), from its origins in **Statistical Machine Translation (SMT)** to the current advancements in **Neural Machine Translation (NMT)**. Our primary focus is on understanding the architectural progression of these models. We will empirically evaluate key MT models using a specific dataset[1], examining approaches from traditional SMT to modern NMT systems. Furthermore, we explore the capabilities of large language models (LLMs) in MT, specifically through **fine-tuning T5**[11] and investigating the performance of **state-of-the-art multimodal large language models (LMMs) like Gemma 3**[8].

1 Task description/Problem statement

The task of converting a piece of text from a source language into a target language, minimizing the semantic distance between the original and translated text while maintaining grammatical correctness and fluency in the target language, is known as machine translation. Most research papers in the domain of NMT (Neural Machine Translation) have been released in the last few years, also thanks to the new Transformers architecture and the increase in the available computational power. In fact, the majority of recent works are based on models like BERT and its successors. Still, previously there were non-neural strategies such as Statistical Machine Translation (SMT), which could still achieve a decent result in this task.

While the topology of Machine Translation is pretty detailed, we noticed that the main distinction that researchers generally make is between statistical and neural approaches. The statistical approach uses probabilistic models learned from parallel corpora, while the latter aims to produce translations by learning complex patterns and representations from vast amounts of data, simulating a more natural human translation behavior.

1.1 Examples

To better explain what Machine Translation aims to be capable of, here we report a few samples from some of the most famous available datasets.

1.1.1 From WMT (Workshop on Statistical Machine Translation)

The WMT datasets are widely used benchmarks in machine translation, providing parallel corpora for various language pairs, often sourced from news commentaries, parliamentary proceedings, and web crawls.

An extract of the article text (English Source):

> ”The European Union is committed to supporting Ukraine's sovereignty and territorial integrity.”

Output translation (German Target):

> ”Die Europäische Union ist entschlossen, die Souveränität und territoriale Integrität der Ukraine zu unterstützen.”

1.1.2 From MultiUN dataset

This dataset contains parallel texts from the official proceedings of the United Nations, covering a wide range of topics and exhibiting formal language.

An extract of the article text (French Source):

> ”Les délégations ont exprimé leur satisfaction quant aux progrès réalisés dans les négociations.”

Output translation (English Target):

”The delegations expressed their satisfaction with the progress made in the negotiations.”

1.1.3 From Anki dataset

The Anki dataset typically contains various language pairs for flashcard-based learning, often featuring shorter, more conversational sentences.

An extract of the article text (English Source):

”Tom noticed that Mary was asleep.”

Output translation (Italian Target):

”Tom notò che Mary era addormentata.”

1.2 Real-world applications

The core idea behind Machine Translation (MT) is to bridge language barriers by automatically converting text or speech from one natural language into another. This capability allows individuals and organizations to communicate and access information across different linguistic communities, significantly expanding reach and understanding.

Daily life applications include:

- **Travel and Tourism:** Enabling tourists to understand local signs, menus, and communicate with locals through translation apps.
- **International Business and E-commerce:** Facilitating communication between international partners and translating product descriptions, reviews, and customer support for global markets.
- **Cross-lingual Communication:** Allowing individuals to communicate with friends, family, or colleagues who speak different languages via messaging apps or social media platforms with integrated translation features.
- **Access to Information:** Providing non-native speakers with the ability to read news articles, research papers, and websites in their native language, thereby democratizing access to global knowledge.
- **Customer Support:** Automating the translation of customer queries and responses, enabling companies to offer support in multiple languages without extensive human resources.

These applications are frequently integrated into broader automated systems. For example, modern web browsers often include built-in MT functionalities to automatically translate entire webpages. Similarly, e-commerce platforms leverage MT to present product information in various languages, and global communication tools frequently offer real-time translation during video calls or chats.

It is generally acknowledged that while immensely beneficial, machine translation also inherently possesses certain limitations. The most significant disadvantage is the potential for inaccuracies, particularly with idiomatic expressions, cultural nuances, or highly technical jargon, which can lead to misinterpretations. Furthermore, the quality of MT can vary significantly depending on the language pair, the domain of the text, and the complexity of the original content. Relying solely on MT, especially for critical or sensitive information, can sometimes result in a loss of the original message’s full intent or subtlety.

2 Related work

2.1 Statistical Machine Translation

The first approach we will analyze is Statistical Machine Translation (SMT). While early ideas about machine translation, including hints at statistical methods for decoding and understanding language, were put forth by Warren Weaver in his influential 1949 memorandum [15], SMT as a distinct paradigm truly emerged and gained prominence later, particularly in the late 20th century. Weaver’s basic idea behind this approach was that language could be treated in the same way as logical mathematics. This contrasted with the rule-based approaches to machine translation, which were the prevalent methods used up to that time.

Statistical Machine Translation operates on the principle of using probabilistic models to translate text. Instead of relying on explicit linguistic rules defined by human experts (as in Rule-Based Machine Translation), SMT systems learn translation patterns by analyzing large amounts of existing human-translated texts, known as parallel corpora. The core idea is to find the most probable translation e (in the target language) given a source sentence f (in the source language), represented by the noisy-channel model:

$$P(e|f) = P(f|e) \times P(e)$$

where $P(f|e)$ is the **translation model** (how likely is the source sentence given the target sentence) and $P(e)$ is the **language model** (how fluent or probable is the target sentence itself).

This problem can be approached by applying **Bayes' Theorem**, where we seek to find the target sentence $E = e_1, e_2, e_3, \ldots, e_n$ that maximizes the probability $P(E|F)$, given the source sentence F:

$$E = \arg\max_E P(E|F)$$

Using Bayes' Theorem, this can be rewritten as:

$$E = \arg\max_E \frac{P(F|E)P(E)}{P(F)}$$

Since $P(F)$ is constant for a given source sentence, the maximization simplifies to:

$$E = \arg\max_E P(F|E)P(E)$$

For a rigorous implementation, one would have to perform an exhaustive search by going through all strings e in the target language. Performing the search efficiently is the work of a machine translation decoder that uses the foreign string, heuristics, and other methods to limit the search space and at the same time keep acceptable quality.

SMT represented a significant advancement over RBMT due to its ability to learn from real-world data, leading to more natural and contextually appropriate translations. It was also more adaptable to new domains by training on relevant corpora.

Despite these limitations, SMT dominated the field of machine translation for many years before the rise of neural approaches.

2.1.1 IBM Models for Word Alignment

A foundational aspect of Statistical Machine Translation, particularly in learning the translation model, is word alignment. The IBM Models[4], developed in the early 1990s, provided a series of increasingly sophisticated probabilistic models for this task. These models aim to determine the correspondence between words in a source sentence and words in its translated target sentence.

IBM Model 1 IBM Model 1 is the simplest of the IBM translation models. Its core idea is to estimate the probability $P(t_j|s_i)$, which represents the probability that a target word t_j is a translation of a source word s_i. A key simplifying assumption in Model 1 is that each target word is generated independently from *exactly one* source word, and that all possible alignments are equally probable. This means the model does not consider the position of words within the sentence when determining alignment.

The primary problem with IBM Model 1 is its disregard for word position. This implies that the probability $P(t_j|s_i)$ is independent of where t_j or s_i appear in their respective sentences. Consequently, Model 1 struggles to account for differences in word order or reordering phenomena between source and target languages, leading to less accurate alignments and, by extension, less fluent translations.

IBM Model 2 IBM Model 2 builds upon Model 1 by introducing a dependence on position. It addresses the fundamental flaw of Model 1 by incorporating an explicit alignment probability. In addition to the translation probability $P(t_j|s_i)$, Model 2 also estimates $P(j|i, l_s, l_t)$, which is the probability that the word at position j in the target sentence aligns to the word at position i in the source sentence, given the lengths of the source sentence (l_s) and the target sentence (l_t).

By adding this positional alignment probability, IBM Model 2 can capture preferences for certain alignment patterns and model word reordering more effectively. This allows it to produce more realistic and accurate word alignments compared to Model 1, as it acknowledges that words are not randomly aligned but rather have a higher likelihood of aligning to words in nearby positions, considering sentence lengths.

2.2 Gated Recurrent Units (GRUs)

As the field of machine translation evolved, a significant shift occurred with the advent of Neural Machine Translation (NMT). A key component of many recurrent neural networks (RNNs) used in NMT are Gated Recurrent Units (GRUs), introduced by Cho et al. in their 2014 paper, "Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation" [3]. GRUs are a type of gating mechanism in recurrent neural networks, designed to address the vanishing gradient problem inherent in vanilla RNNs, similar to Long Short-Term Memory (LSTM) units. The vanishing gradient problem makes it difficult for RNNs to learn long-term dependencies in sequential data.

GRUs achieve this by using two main "gates" that control the flow of information:

- **Update Gate (z_t):** This gate determines how much of the previous hidden state should be carried over to the current hidden state. It helps the model decide how much past information to keep.
- **Reset Gate (r_t):** This gate determines how much of the previous hidden state should be forgotten. It allows the model to drop information that is irrelevant to the future.

The hidden state calculation for a GRU at time t can be expressed as:

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

where $\tilde{h}_t$ is the candidate hidden state, calculated using the reset gate:

$$\tilde{h}_t = \tanh(W_h x_t + U_h(r_t \odot h_{t-1}))$$

Here, W_h and U_h are weight matrices, x_t is the input at time t, and $\odot$ denotes element-wise multiplication.

The simpler design of GRUs, by integrating the cell state and hidden state, and merging the input and forget gates, streamlines the learning process while still effectively capturing long-range dependencies. This efficiency and often comparable performance make them a powerful tool in recurrent neural networks, contributing significantly to the advancements in Neural Machine Translation presented in the paper.

2.3 The Attention Mechanism

Traditional Recurrent Neural Networks (RNNs), especially basic encoder-decoder architectures, struggle to effectively process long sequences. A common problem is that they are forced to compress all the information from the input sequence into a single fixed-size context vector (the final hidden state of the encoder). This often leads to a bottleneck, where relevant information from earlier parts of the sequence can be forgotten or diluted, making it difficult for the decoder to generate accurate outputs, particularly for lengthy sentences. This is because the assumption that the last input summarizes sufficient information to generate the output is often false; in many scenarios, the last part of a sequence is not the most important part.

The **Attention Mechanism** was introduced to solve this problem by allowing the model to "focus" on different parts of the input sequence when generating each part of the output sequence. Instead of compressing the entire input into a single context vector, attention mechanisms enable the decoder to selectively look at relevant parts of the source sequence at each decoding step.

The core idea involves a trainable "attention module" that calculates relevance coefficients (or "attention weights") for each token in the input sequence with respect to the current decoding step. As described in the provided text, for each token i in the input sequence, a score e_i is calculated:

$$e_i = a(s, h_i)$$

where:

- s is the context (e.g., the decoder's current hidden state from the previous time step, representing what has been generated so far).
- h_i is the hidden state (or representation) of the i-th token from the encoder's output.
- a is a function (which can be implemented with a neural network, e.g., a feed-forward network) that scores how well the input at position i and the output at the current position match.

Once these raw scores (e_i) are obtained, they are normalized, typically using a **softmax function**, to get the attention weights α_i:

$$\alpha_i = \frac{\exp(e_i)}{\sum_j \exp(e_j)}$$

These α_i values represent how much "attention" the model should pay to each input token. Tokens that are more relevant to generating the current output token will have a higher α_i value.

Finally, a new context vector c is computed as a weighted sum of the encoder's hidden states, where the weights are the attention coefficients:

$$c = \sum_i \alpha_i h_i$$

This context vector c is then passed to the decoder along with the previously generated output to help generate the next output token. This dynamic context allows the decoder to "look back" at specific parts of the source sentence, overcoming the fixed-size bottleneck of traditional encoder-decoder RNNs and significantly improving the quality of sequence-to-sequence tasks like machine translation [2].

2.4 Large Language Models (LLMs)

Large Language Models (LLMs) represent a significant leap in natural language processing, characterized by their immense scale, ability to learn from vast amounts of text data, and remarkable generalization capabilities across diverse tasks. These models, often based on the Transformer architecture, have pushed the boundaries of what AI can achieve in understanding, generating, and interacting with human language.

While powerful, LLMs often provide a general understanding of language. For optimal performance in specialized applications, these models frequently need to be tailored to specific contexts. For example, the language and terminology used in medical documents are vastly different from those found in financial reports. A general-purpose LLM might struggle with the nuances, jargon, and specific reasoning patterns required for high accuracy in such specialized domains. Therefore, fine-tuning LLMs on domain-specific data is crucial to adapt them to the target language and context, ensuring that their outputs are precise, relevant, and adhere to the unique characteristics of that field.

2.4.1 Fine-tuning Large Language Models with LoRA

Fine-tuning Large Language Models (LLMs) for specific downstream tasks or domains is crucial to maximize their performance and applicability. However, fine-tuning an entire LLM, especially those with billions of parameters, is computationally expensive and memory-intensive. To address these challenges, **Low-Rank Adaptation (LoRA)** has emerged as a highly efficient fine-tuning technique [6].

LoRA works by freezing the pre-trained model's weights and injecting small, trainable low-rank decomposition matrices into each layer of the Transformer architecture. When fine-tuning, only these newly added low-rank matrices are updated, rather than the entire set of original model parameters.

Specifically, for a pre-trained weight matrix $W_0 \in R^{d \times k}$, LoRA approximates the update ΔW by a low-rank decomposition BA, where $B \in R^{d \times r}$ and $A \in R^{r \times k}$, and $r \ll \min(d, k)$. Thus, the modified forward pass becomes:

$$h = W_0 x + BAx$$

where x is the input and h is the output. Only A and B are updated during training.

The key advantages of using LoRA for fine-tuning LLMs include:

- **Reduced Computational Cost:** By training only a small fraction of the total parameters, LoRA significantly reduces the computational power required for fine-tuning.
- **Lower Memory Usage:** The memory footprint during fine-tuning is drastically reduced, as only the new low-rank matrices and their gradients need to be stored. This makes it possible to fine-tune large models on consumer-grade hardware.
- **Faster Training:** With fewer parameters to update, the fine-tuning process converges much faster.
- **Improved Efficiency for Deployment:** Different fine-tuned adapters (LoRA modules) can be swapped in and out for a single base model without storing multiple full copies of the model, enabling efficient serving of multiple specialized models.
- **Reduced Overfitting:** Training fewer parameters can also help mitigate overfitting, especially when fine-tuning on smaller, task-specific datasets.

LoRA has proven particularly effective for adapting large pre-trained models to various downstream NLP tasks with minimal performance degradation compared to full fine-tuning, making it a pivotal technique in the practical application of LLMs.

2.5 Tower v2 for Neural Machine Translation

Tower v2[14] represents an advanced, open-weight multilingual Large Language Model (LLM) specifically designed and optimized for Neural Machine Translation (NMT) and related tasks. It is an improved iteration of the original Tower models, developed through a collaborative effort by Unbabel and academic partners.

Key aspects of Tower v2 relevant to NMT include:

- **Continued Pre-training (CPT):** An initial phase where the base LLM is further trained on massive multilingual datasets, including both monolingual and parallel corpora. This stage is crucial for enhancing the model's understanding of diverse languages and their relationships, extending beyond the original pre-training of the base model.

- **Supervised Fine-tuning (SFT):** Following CPT, the model is fine-tuned on high-quality, diverse datasets specifically tailored for various translation-related tasks. These tasks encompass not only general machine translation (at both sentence and document levels) but also more nuanced applications like automatic post-editing (APE), grammatical error correction (GEC), and named-entity recognition (NER). The use of curated datasets, such as "TowerBlocks," ensures the model learns to perform these tasks effectively.

- **Competitive Performance:** Notably, Tower v2 has demonstrated highly competitive performance in leading translation benchmarks, such as the WMT24 General Translation shared task. It has even shown to outperform some well-established closed commercial systems (e.g., GPT-4o, Claude 3.5, and DeepL) in certain language pairs, even at smaller scales (e.g., 7B parameters). Its capabilities extend to paragraph- and document-level translation, addressing challenges where previous models might have struggled.

In essence, Tower v2 signifies a substantial advancement in adapting large, general-purpose LLMs to excel within the specialized domain of machine translation, pushing the boundaries of what open-source models can achieve in this field.

3 Datasets and Benchmarks

Machine Translation (MT) research relies heavily on specialized datasets and robust evaluation metrics to develop and compare models. These resources allow researchers to train models on vast amounts of parallel text and assess their translation quality against established benchmarks.

3.1 Datasets

3.1.1 WMT (Workshop on Machine Translation) Datasets

The Workshop on Machine Translation (WMT) annually hosts shared tasks that provide a diverse collection of parallel corpora for various language pairs. These datasets are widely used as benchmarks for evaluating MT systems. They typically include news texts, common crawls, and other domain-specific texts, fostering research on different translation challenges and language pairs.

3.1.2 Europarl

The Europarl corpus is a widely used parallel corpus extracted from the proceedings of the European Parliament [9]. It is available for numerous European language pairs, making it a valuable resource for training and evaluating MT systems, particularly for formal and political domains. Its size and multi-lingual nature make it a cornerstone in MT research.

3.1.3 ParaCrawl

ParaCrawl is a large-scale collection of parallel web-crawled data, providing massive parallel corpora for numerous language pairs [5]. Unlike curated corpora, ParaCrawl leverages web data, offering a broader and often more informal linguistic coverage. Its immense size makes it valuable for training robust data-driven MT systems.

3.2 Evaluation Metrics

Evaluating the quality of machine translation is a complex task, often relying on automated metrics that correlate with human judgments.

3.2.1 BLEU (Bilingual Evaluation Understudy)

BLEU [10] is one of the most widely used metrics for evaluating the quality of machine-translated text. It measures the similarity between a candidate translation and a set of human-created reference translations. BLEU primarily focuses on n-gram overlap, where it counts the number of matching n-grams (sequences of n words) between the candidate and reference translations. It also incorporates a brevity penalty to penalize overly short translations. A higher BLEU score indicates a closer resemblance to the human reference translations.

3.2.2 TER (Translation Error Rate)

TER [13] measures the number of edits (insertions, deletions, substitutions, and shifts of word sequences) required to transform a hypothesis translation into one of the reference translations, normalized by the average length of the references. A lower TER score indicates a better translation, as fewer edits are needed to match the reference.

3.2.3 BERTScore

BERTScore [16] is a more recent evaluation metric that leverages contextual embeddings from pre-trained BERT models (or similar Transformers) to assess the semantic similarity between candidate and reference translations. Instead of relying on exact n-gram matches, BERTScore computes the cosine similarity between the embeddings of words in the candidate and reference sentences. It then aligns and scores the tokens based on these similarities, providing a more nuanced understanding of semantic equivalence. This often leads to a stronger correlation with human judgments compared to traditional n-gram based metrics, especially for abstractive translations.

4 Existing Tools, Libraries, and Reproducibility

The field of **Neural Machine Translation (NMT)** has seen a rapid proliferation of research, leading to a rich ecosystem of publicly available code, implementations, and testing frameworks. While many researchers strive for reproducibility and openly share their work, it is important to note that the most cutting-edge models are often not publicly released due to significant economic and competitive interests.

A common challenge in working with advanced NMT models is their substantial computational resource requirements. Training or fine-tuning these models often necessitates prolonged execution on high-performance GPUs, such as an NVIDIA L4 or even more powerful accelerators. Furthermore, model size directly correlates with performance, implying that larger models, while yielding superior results, demand considerable GPU memory (often tens of gigabytes) to accommodate their parameters, intermediate activations, and optimizer states.

Fortunately, libraries like `HuggingFace Transformers` mitigate some of these challenges. This invaluable resource provides access to a vast collection of pre-trained NMT models across numerous languages and architectures. These models can be readily loaded and fine-tuned on custom datasets, significantly reducing the initial computational burden and development time.

Evaluation Metrics and Tools

For the evaluation of NMT systems, several robust libraries are available. Specifically, we are interested in the widely adopted **BLEU** (Bilingual Evaluation Understudy) and **SacreBLEU** metrics. Implementations of these metrics are readily accessible through libraries such as `NLTK` and the `HuggingFace 'evaluate' library`.

It is crucial to acknowledge that while various libraries offer BLEU computations, minor discrepancies can arise due to differences in underlying tokenization schemes, smoothing methods, and handling of the brevity penalty. To ensure the comparability and reproducibility of our results, we primarily rely on the `sacrebleu` implementation from the `evaluate` library. SacreBLEU is specifically designed to provide standardized and reproducible scores by adhering to established evaluation protocols, including consistent tokenization and transparent version reporting.

Datasets and Resource Constraints

For our experiments, various **WMT** are available. While these datasets are invaluable for NMT research, they often contain hundreds of millions of parallel sentences, posing significant challenges for local training or even extensive experimentation without substantial GPU resources. The sheer scale of these datasets makes it practically impossible to fully train a model on a personal machine within a reasonable timeframe. Consequently, we have opted to utilize smaller, more manageable subsets of these datasets. This approach allows for partial training and testing on local machines, with full training and hyperparameter tuning reserved for cloud-based GPU environments like Google Colab. This pragmatic decision balances the need for robust experimentation with available computational resources.

5 State-of-the-Art Evaluation in Neural Machine Translation

The competitive landscape of Neural Machine Translation (NMT) is vividly showcased by the annual **Workshop on Machine Translation (WMT)** shared tasks, which serve as crucial benchmarks for assessing state-of-the-art systems. The WMT 2024 General Machine Translation Shared Task provided compelling evidence of advancements, particularly highlighting the performance of open-source models against prominent commercial, closed-source systems.

In WMT 2024, the top performer was **Tower v2** [14], an impressive open-weight model from Unbabel-IST. Tower v2 demonstrated superior translation quality compared to powerful closed-source models such as GPT-4o, Claude 3.5, and DeepL. This remarkable achievement was primarily realized through meticulous fine-tuning of existing open-source Large Language Models (LLMs), specifically Mistral-7B (7 billion parameters) and Llama-3-70B (70 billion parameters).

The evaluation of these systems predominantly relied on advanced automatic metrics, notably **COMET** [12] and **MetricX** [7]. These metrics, which correlate strongly with human judgments, confirmed Tower v2's lead. Beyond automatic scores, human evaluation further validated these findings: Tower v2 surpassed other models in 8 out of 11 evaluated language pairs, signifying its robust performance across a diverse linguistic spectrum.

This outcome underscores a significant trend: while general-purpose LLMs like GPT-4o exhibit strong inherent translation capabilities even in a **zero-shot learning** setting (i.e., without explicit fine-tuning for translation), fine-tuned open-source models can still achieve superior specialized performance. Although closed LLMs demonstrated improved results in **few-shot learning** scenarios for certain languages (e.g., Hindi), they did not ultimately outperform Tower v2. This highlights the continued importance of task-specific adaptation and high-quality fine-tuning data in achieving true state-of-the-art results in NMT, even in an era dominated by large foundational models.

6 Comparative evaluation

6.1 Dataset

To run this comparative evaluation i decided to take a medium size dataset with **354238** records formatted as `(english sentence, english sentence translated to italian)`, and is divided in 60% for train set, 20% for validation and 20% for test. another good thing (for performance) is that the 97% of the sentences are relativly short (10 or less words per sentence). this also introduce a bias considering that probably it will perform better for shour sentences in case of neural networks.

6.2 Models Evaluation

I have evaluated the following models

1. **IBM1 Model**: Statistical Machine Translation model. Implemented from scratch and trying to minimize the memory allocation, and implemented with Expectation Maximization pattern.

2. **GRU**: Sequence-to-sequence model with encoder-decoder architecture and attention weights. Implemented from Scratch with the GRU implementation of skitlearn.

3. **T5**: Ecoder-Decoder Transformer, trained with instruction it, it was taken from the transformers Libriry of Hugging's face

4. **Gemma3**: Pretrained model used with instruction with and without finetuning. It was taken from the libriry of hugging face and finetuned with LoRa.

6.3 Metrics and evaluation protocol

Different evaluation protocols were adopted depending on the model size and computational constraints. For large models such as **Gemma**, the evaluation dataset was limited to 10,000 samples instead of using the entire corpus. This decision was primarily motivated by the restricted computational resources available (e.g., limited Google Colab credits).

For the **Gemma 3** model, a specific prompt was used:

> *Translate from English to Italian (just the sentence):*

The rationale for using this exact formulation will be discussed in a later section.

In the case of the **T5** model, instruction fine-tuning was necessary. The model was trained to recognize the instruction prefix:

Translate from English to Italian

allowing it to adapt to the translation task during fine-tuning.

The **GRU-based** model was also trained with computational limitations in mind. After observing that the model began converging after more than 20 epochs with a decreasing loss trend, the training dataset was extended to 100,000 samples.

Ideally, a fair evaluation would involve using the same underlying data split (train, validation, test) for all models, sliced consistently from the full dataset. However, this was not strictly enforced. Instead, randomized sampling was adopted to reduce possible biases—such as the first samples being overly similar or unusually long—which could affect model performance.

These decisions, while pragmatic, introduce potential variability in evaluation fairness and should be considered when comparing results across models.

For the evaluation of translation quality, I chose to use only the **BLEU** metric. While alternative metrics such as **COMET** and **MetricX** are widely adopted in recent Neural Machine Translation (NMT) literature, they were not used in this work due to practical constraints related to the dataset format and availability.

In particular, **COMET** is designed to evaluate translations by considering multiple valid reference variations (e.g., the English word *run* might be accurately translated as either *corri*, *corro*, or other contextually appropriate forms). However, the dataset used in this project does not support multiple gold references or the structure required for COMET-style evaluation.

On the other hand, **BLEU** provides a more rigid, string-based comparison and treats semantically equivalent but lexically different translations as distinct (e.g., *corri* and *corro* would be penalized as mismatches). Despite its limitations, BLEU remains a standard and reproducible metric and was thus selected for consistency and feasibility in this context.

6.4 Results

The results are presented in the Table 1.

—

For **IBM1**, the BLEU score is low. By examining some of the results, it's easy to conclude that the main issue is alignment. What it currently does is merely map each English word to an Italian one without considering order or other linguistic nuances. This problem was addressed in IBM Model 2, which also predicts the mapping between input words and the position of output words, and would achieve better results. However, IBM Model 2 wasn't possible to implement considering the dimentions of the tables. Currently, we already have a table of approximately $25,000 \times 12,000$, which accounts for about 3^8 of memory usage. Considering that word representation is already an optimization over string representation, my approach is to assign an integer ID to each word to save space and facilitate better retrieval.

The **GRU** implementation was quite impressive, especially considering that I hit the termination condition at epoch 17. Given the number of available samples, it was a good result. During training, the teacher forcing strategy was used to prevent the model from continuing in the wrong direction, encouraging it to learn even after making a mistake with a word.

For **T5 (small)**, the results were the most unexpected. The reason behind this seems to be that it didn't truly understand the instruction, leading it to translate from English to Spanish, or to add English words to the Italian translations. In some cases, the Italian translations also lacked coherence, making it a significant flop. Perhaps mT5, which is already fine-tuned for a variety of tasks, would perform better, even if overall it achived a good result.

For **Gemma 3 1B**, I had very high expectations, and it delivered. When I tested it, it provided many good answers even without fine-tuning. The prompt also played a significant role, and I tried to find the best prompt by weighing its length (shorter length $\rightarrow$ fewer tokens $\rightarrow$ faster inference) and the type of response. For example, with some prompts, it would respond like "if you want X then Y, if you want X' then Y'". In some cases, it also attempted to provide translations for both masculine and feminine by adding "a/o" to some words. I believe that some of these issues are generally resolved by fine-tuning, and that this "low" BLEU score is solely due to the limitations of BLEU. Perhaps with COMET or human verifica-

Table 1: Model Performance on English-to-Italian Translation

Model	BLEU Score	Dataset Size	Fine-Tuning / Prompt
Gemma 3 1B	0.19	10,000	Prompt-based (zero-shot)
Gemma 3 1B fine-tuned	0.24	10,000	Prompt-based (zero-shot)
T5 (small)	0.20	100,000	Instruction fine-tuned
GRU-based	0.41	85,000	Trained from scratch
IBM1	0.10	350,000	Trained from scratch

tion, it would receive higher scores, considering that it translates very well even in instances where it receives a BLEU score of 0.

6.5 Discussion

In this survey, I explored various systems that are and have been state-of-the-art models in machine translation. I gained a deeper understanding of their computational limitations. For instance, IBM1 faces challenges in calculating likelihoods, while GRU models require waiting for the previous state before generating values. In contrast, modern architectures, particularly transformer-based models, are significantly more efficient due to their parallelization capabilities, leading to excellent performance.

However, these advanced models can also generate diverse solutions that, while semantically correct, may differ from expected reference translations. This directly highlights the limitations of the BLEU metric, emphasizing the importance of human feedback or more flexible metrics like COMET, which can accommodate multiple valid translations. Adopting such metrics, however, necessitates a dataset that supports this type of evaluation.

Regarding future work, it would be interesting to investigate how the Tower v2 model, a current state-of-the-art architecture, performs when integrated with the Gemma 3 family. This could be achieved by following the methodologies outlined in their respective papers, though it would require access to additional GPU resources. Furthermore, it is crucial to understand the comparative impact of COMET versus BLEU in the evaluation of Large Language Models (LLMs). A significant challenge lies in developing methods to generate or augment datasets with the necessary labels for COMET evaluation, especially for existing datasets that currently lack such annotations. This is similar to the creation of new, synthetically generated data.

6.6 Code

Github repository

References

[1] Anki datasets. https://www.manythings.org/anki/ita-eng.zip. 1

[2] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. *arXiv preprint arXiv:1409.0473*, 2014. 5

[3] Kyunghyun Cho, Bart Van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using rnn encoder-decoder for statistical machine translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1724–1734, 2014. 3

[4] Michael Collins. Statistical machine translation: Ibm models 1 and 2, 2016. Stanford University CS224N Handout. 3

[5] Miquel Esplà-Gomis. ParaCrawl: A Massive Collection of Parallel Web-Crawled Data. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 2493–2498, 2019. 6

[6] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021. 5

[7] Juraj Juraska, Daniel Deutsch, Mara Finkelstein, and Markus Freitag. MetricX-24: The Google Submission to the WMT 2024 Metrics Shared Task. *arXiv preprint arXiv:2410.03983*, 2024. 8

[8] Aishwarya Kamath, Johan Ferret, Shreya Pathak, Nino Vieillard, Ramona Merhej, Sarah Perrin, Tatiana Matejovicova, Alexandre Ramé, Morgane Rivière, Louis Rouillard, Thomas Mesnard, Geoffrey Cideron, Jean bastien Grill, Sabela Ramos, Edouard Yvinec, Michelle Casbon, Etienne Pot, Ivo Penchev, Gaël Liu, Francesco Visin, Kathleen Kenealy, Lucas Beyer, Xiaohai Zhai, Anton Tsitsulin, Robert Busa-Fekete, Alex Feng, Noveen Sachdeva, Benjamin Coleman, Yi Gao, Basil Mustafa, Iain Barr, Emilio Parisotto, David Tian, Matan Eyal, Colin Cherry, Jan-Thorsten Peter, Danila Sinopalnikov, Surya Bhupatiraju, Rishabh Agarwal, Mehran Kazemi, Dan Malkin, Ravin Kumar, David Vilar, Idan Brusilovsky, Jiaming Luo, Andreas Steiner, Abe Friesen, Abhanshu Sharma, Abheesht Sharma, Adi Mayrav Gilady, Adrian Goedeckemeyer, Alaa Saade, Alex Feng, Alexander Kolesnikov, Alexei Bendebury, Alvin Abdagic, Amit Vadi, András György, André Susano Pinto, Anil Das, Ankur Bapna, Antoine Miech, Antoine Yang, Antonia Paterson, Ashish Shenoy, Ayan Chakrabarti, Bilal Piot, Bo Wu, Bobak Shahriari, Bryce Petrini, Charlie Chen, Charline Le Lan, Christopher A. Choquette-Choo, CJ Carey, Cormac Brick, Daniel Deutsch, Danielle Eisenbud, Dee Cattle, Derek Cheng, Dimitris Paparas, Divyashree Shivakumar Sreepathihalli, Doug Reid, Dustin Tran, Dustin Zelle, Eric Noland, Erwin Huizenga, Eugene Kharitonov, Frederick Liu, Gagik Amirkhanyan, Glenn Cameron, Hadi Hashemi, Hanna Klimczak-Plucińska, Harman Singh, Harsh Mehta, Harshal Tushar Lehri, Hussein Hazimeh, Ian Ballantyne, Idan Szpektor, Ivan Nardini, and Gemma Team. Gemma 3 technical report. *arXiv preprint arXiv:2503.19786*, 2025. 1

[9] Philipp Koehn. Europarl: A Parallel Corpus for Statistical Machine Translation. In *Proceedings of the Tenth Machine Translation Summit*, pages 79–86, 2005. 6

[10] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. BLEU: a Method for Automatic Evaluation of Machine Translation. In *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, pages 311–318, 2002. 6

[11] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140):1–67, 2020. 1

[12] Ricardo Rei, Craig Stewart, Marina Fomicheva, Ana C Kasner, Grzegorz Märchler, Alissa Riza, and Dimitar Shterionov. Comet: A neural framework for mt evaluation. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 5352–5368, 2020. 8

[13] Matthew Snover, Bonnie Dorr, Richard Schwartz, Linnea Micciulla, and John Makhoul. TER: A Metric for Machine Translation Evaluation that Favors Fewer Edits. In *Proceedings of the International Workshop on Spoken Language Translation (IWSLT)*, pages 216–219, 2006. 7

[14] Unbabel Research Team and IST Lisbon. TOWER: Scaling Seq2Seq LLMs for High-Quality Machine Translation. In *Proceedings of the Fifth Conference on Machine Translation (WMT24)*, 2024. 5, 8

[15] Warren Weaver. Machine translation of languages. Technical report, Rockefeller Foundation, 1949. 2

[16] Tianyi Zhang, Denis Kneer, Zhenzhou Zhang, Yang Du, Zihan Liu, Yiqing Lu, Zichao Liu, Zifeng Huang, Wei Zhao, Xinyu Li, et al. BERTScore: Evaluating Text Generation with BERT. *arXiv preprint arXiv:1904.09458*, 2019. 7